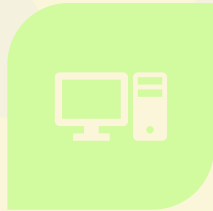


Evaluating Open-Source LLMs for Code Validation

Yasteel Ramphal
Ziphokazizamagcugcwa Nzama
Mohemad Kudia



Introduction – Background and Context



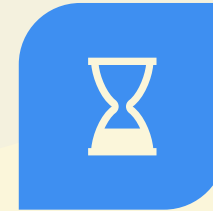
**THE RISE OF
LARGE
LANGUAGE
MODELS IN
SECURE
SOFTWARE
DEVELOPMENT**



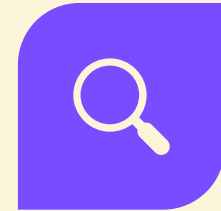
**TRANSFORMING
SOFTWARE
ENGINEERING**
CUTTING-EDGE
LLMS
REVOLUTIONIZE
CODE WRITING,
REVIEWING, AND
DEBUGGING



**THE CRITICAL
ROLE OF CODE
REVIEW**
ESSENTIAL FOR
CATCHING BUGS,
IMPROVING
MAINTAINABILITY,
AND
SAFEGUARDING
SECURITY



THE CHALLENGE
MANUAL REVIEWS
ARE TIME-
CONSUMING,
ERROR-PRONE,
AND REQUIRE
DEEP EXPERTISE



**OPEN-SOURCE
LLMS: A NEW
FRONTIER**
TRANSPARENCY,
CUSTOMIZATION,
AND
ACCESSIBILITY
DRIVE INTEREST –
YET THEIR
SECURITY
POTENTIAL
REMAINS LARGELY
UNEXPLORED

Introduction – Problem Statement

Key Challenges for Open-Source LLMs

- Can they accurately flag **security vulnerabilities**?
- Are they capable of **functional validation** for correctness?
- Can one model handle **security and correctness simultaneously**?
- Do they provide **actionable feedback** developers can trust?

Why This Matters:

Security flaws often stem from **logical errors** & **faulty implementations** – highlighting the critical link between **functionality** and **security**.



Introduction: Research Objectives

This study systematically evaluates **open-source LLMs** on:

- 1.Security Vulnerability Detection** – Identify common flaws in code
- 2.Functional Validation** – Detect errors impacting software behavior
- 3.Prompting Techniques Comparison** – Zero-shot vs. Chain-of-Thought
- 4.Quality of Feedback** – Accuracy & usefulness of vulnerability descriptions
- 5.Integration Strategies** – Embedding LLMs into developer workflows

Related Work

Code Review Challenges

From formal to informal, tool-assisted reviews
Limited review time; focus on design, not exhaustive checks
High contribution volume strains quality
Static analysis tools cause many false positives

LLMs in Software Engineering

Performance varies by language and task complexity
Fine-tuned code models outperform general LLMs
Challenges: multi-step reasoning, logical errors, explainability
Research mostly on proprietary models

Security Vulnerability Detection

Promising but detecting subtle vulnerabilities remains difficult
Conversational AI models help explain vulnerabilities (mostly closed-source)
Logical and multi-file flaws hardest to detect
Specialized models outperform general-purpose ones
Open-source models' effectiveness remains unclear

Functional Validation Approaches

Effective for simple checks; struggles with complex algorithms
Self-debugging prompts improve accuracy
Hybrid methods combining LLMs and traditional verification best
Key: prompt design, task decomposition, hybrid workflows

Methodology

Research Design & Approach

Research Framework

- ▶ Systematic experimental approach
- ▶ Factorial structure design
- ▶ Controlled comparisons across models
- ▶ Standardized evaluation protocols

Key Variables

- ▶ **Independent:** Model type & prompting technique
- ▶ **Dependent:** Detection accuracy, validation accuracy, explanation quality
- ▶ **Controlled:** Code complexity, vulnerability type

Evaluation Tasks

- ▶ Security vulnerability detection
- ▶ Functional validation
- ▶ Combined assessment

Model Selection

Five Open-Source LLMs from Hugging Face Leaderboard

Viper-Coder-v1.6

~14B parameters
High-capacity model
Strong in complex vulnerabilities

Llama-3.1-SuperNova-Lite

8B parameters
LLaMA architecture
Balanced accuracy & resources

AceCoder-Qwen2.5-7B

7.6B parameters
Qwen2.5 series
Consistent instruction-following

Llama-3.2-3B-Agent007

4B parameters
Lightweight model
Suitable for simpler tasks

Qwen2.5-Coder-0.5B

0.5B parameters
Smallest model
Low-resource optimization

Selection Criteria: Performance in programming tasks, architectural diversity, license compatibility, and practical software engineering relevance

Dataset & Prompting Techniques

OWASP Benchmark for Java

- ▶ Comprehensive Java code samples
- ▶ Explicitly labeled vulnerable/safe
- ▶ Categories: SQL injection, XSS, path traversal
- ▶ Real-world security patterns

Evaluation Metrics

- ▶ **Precision, Recall, F1-score**
- ▶ Detection accuracy
- ▶ Explanation quality
- ▶ Response time & resource usage

Zero-Shot Prompting

- ▶ Clear task instructions
- ▶ Expected output format
- ▶ No reasoning examples
- ▶ Tests inherent capabilities

Chain-of-Thought Prompting

- ▶ Structured reasoning process
- ▶ Step-by-step guidance
- ▶ Explicit analysis framework
- ▶ Enhanced complex reasoning

Overview of LLM Evaluation Implementation

Exploring the core components of LLM performance and security assessments



Platform: Google Colab

Utilising **Google Colab** with **CUDA GPU support** for computation.



Framework: Python-based

Utilising a **Python-based evaluation system** to facilitate rigorous LLM testing and analysis.



Models: 5 open-source LLMs

Incorporating **5 open-source LLMs** ranging from **0.5B to 14B parameters** for diverse performance metrics



Test Suite: Security & Functional

Developing a **test suite** that includes both **security vulnerabilities** and **functional validation** measures



Evaluation: Zero-shot vs Chain-shot

Implementing **evaluation methods** comparing **zero-shot** and **chain-of-thought prompting** techniques



Core Components Overview

Key components include **model loading**, **memory management**, and **automated test execution** for efficiency.

Optimizing Model Loading with Quantization

Exploring 4-Bit Quantization for Large Models



4-Bit Quantization Configuration

Reduces model precision from 32-bit to 4-bit numbers, decreasing memory requirements while maintaining acceptable performance levels.



Memory Reduction

The quantisation technique significantly cuts GPU memory consumption, enabling large model deployment on standard hardware that has limited resources.



Model Accessibility

Memory optimisation allows users to run large 14-billion parameter models on limited hardware that previously couldn't support them (limitations of colab enviromen).



Automatic Device Mapping

System automatically distributes model components across available GPU's and CPU's to maximize performance and prevent memory overflow issues.

Pipeline and Tokenisation Process



Inference Mode Efficiency

Inference Mode disables gradient computation to enhance efficiency during model execution, allowing for faster processing times and reduced resource usage.



Temperature Control Setting

Setting the **temperature** to **0.2** provides reproducible results by controlling randomness in predictions, leading to stable outputs.



Token Limits Control

The maximum limit of **250 tokens** ensures consistent comparison across different model outputs, facilitating a fair evaluation of performance.



Memory-Efficient Cleanup

Automatic cleanup after each inference ensures **memory efficiency**, preventing unnecessary memory usage and optimising res. Clearing model from system once completed

Exploring Dual Prompting Techniques

Understanding Zero-Shot and Chain-of-Thought Prompting



Zero-Shot Prompting

Zero-shot prompting involves making predictions without any prior examples or training on the specific task, leveraging existing knowledge to generate responses.



Chain-of-Thought Prompting

Chain-of-thought prompting encourages the model to reason step-by-step, improving performance on complex tasks by breaking them down into manageable components.

Outputs are classified in a confusion matrix according to true and false positives and negatives to analyse LLM's accuracy

```
prompts = {
    "zero_shot": f"""Identify where in this code the security issues and functional issues are.
    {code}

    Identify security vulnerabilities:
    Identify functional issues:""",

    "chain_of_thought": f"""Analyze this code step-by-step:
    1. Identify the security vulnerabilities
    2. Identify the functional validation failures
    3. Explain reasoning for each finding
    4. Provide the corrected code if it requires it

    Code:
    {code}

    Analysis:"""
}

full_prompt = prompts[prompt_type]

resource_usage_before = get_resource_usage()

input_tokens = len(tokenizer.encode(full_prompt))
start_time = time.time()

# Convert input to LLM input format -mapping
with torch.inference_mode():
    pipe = pipeline(
        task="text-generation",
        model=model,
        tokenizer=tokenizer,
        max_new_tokens=250, # Increased token limit for more detailed responses
        temperature=0.2, # Limit randomness for reproducibility
        repetition_penalty=1.2
    )

    result = pipe(prompts[prompt_type], return_full_text=False)
    generated_text = result[0]['generated_text']

    response_time = time.time() - start_time

    output_tokens = len(tokenizer.encode(generated_text))
```

Key Metrics for Resource Monitoring

LLM's resource performance metrics are assessed against Response time, token usage and Memory usage



Response Time

This metric indicates the **end-to-end inference duration**, measuring how quickly the system responds to requests during evaluation.



Token Throughput

Refers to the **tokens generated per second**, a crucial measure of the system's efficiency in processing data.



Memory Usage

Tracks the **CPU and GPU allocation**, providing insights into the resource consumption and efficiency of the system during evaluation.

Results : Security Vulnerability Detection Performance



- Prompting Impact:**

Chain-of-thought improved accuracy for EpsteinLlama (44% → 78%) and vonJack (22% → 78%).

- Performance Variation:**

Zero-shot accuracy ranged from 22% (vonJack) to 78% (AceCoder, Viper).

- False Positives:**

All models reported 2 consistent false positives, regardless of prompting.

- False Negatives:**

Chain-of-thought eliminated false negatives in CodeLlama, EpsteinLlama, and vonJack (recall → 100%).

- Top Performers:**

Most models converged at 78% accuracy with identical error patterns (7 TP, 2 FP, 0 FN).

Results :

Functional Validation Performance

Strong Overall Performance:

Accuracy ranged from 89% to 100%, showing effective functional issue detection.

Top Models:

AceCoder and Viper achieved 100% accuracy with zero-shot prompting.

Prompting Impact:

Chain-of-thought improved EpsteinLlama from 89% to 100%, removing its single false negative.

Consistent Errors:

CodeLlama and vonJack stayed at 89% with identical errors (8 TP, 1 FN) regardless of prompting.

No False Positives:

All models showed high precision with zero false positives in functional validation.

Results : Combined Task Performance

Combined Performance:

Accuracy ranges from 56% (vonJack ZS) to 89% (AceCoder, Viper)

Chain-of-thought boosts EpsteinLlama (67%→89%), vonJack (56%→83%), CodeLlama (78%→83%)

Recall improved notably across models; e.g., EpsteinLlama to 100%

Viper and AceCoder perform consistently, unaffected by prompting

F1 scores rise with chain-of-thought for EpsteinLlama and vonJack

Efficiency & Resource Use:

vonJack fastest & lightest: 10s response, 0.02GB GPU, but lower accuracy (83%)

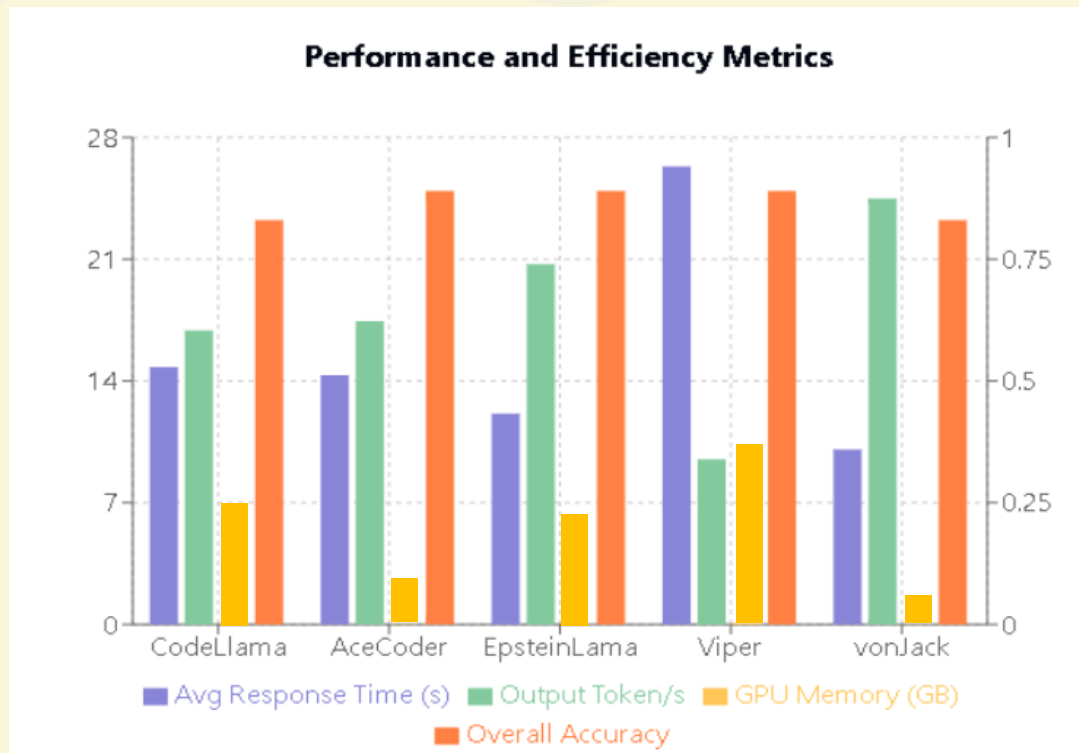
Viper most accurate (89%) but slowest and highest resource use

EpsteinLlama & AceCoder balanced speed and accuracy

Smaller models trade robustness for speed; larger models vice versa

Output length similar (~4500 tokens) across all models

Performance Trade-offs and Resource Efficiency



Speed vs. Accuracy Trade-offs

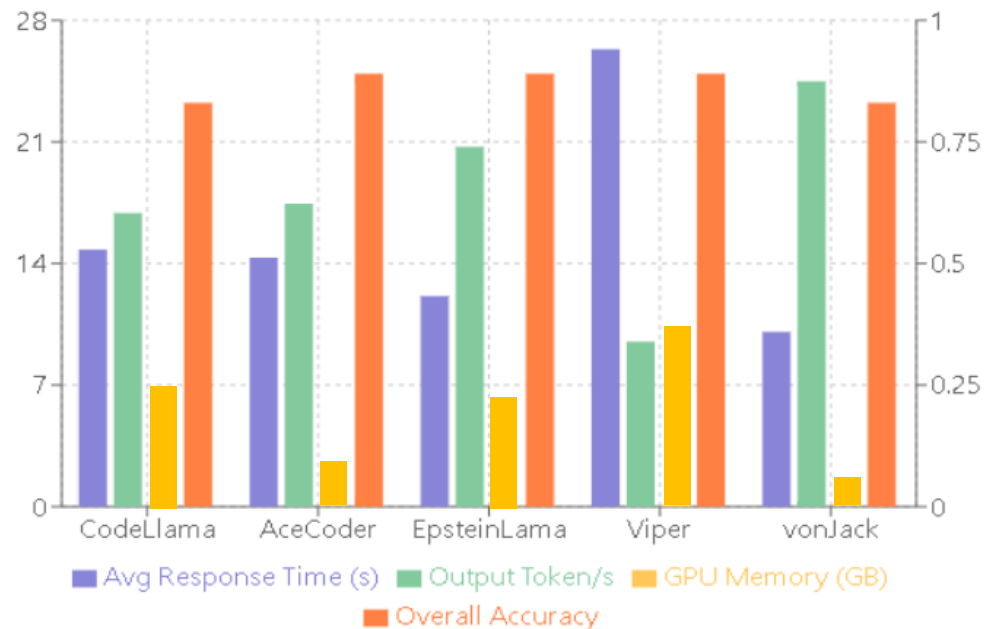
- **Speed** : vonJack delivers fastest performance (10.06s response) with minimal memory footprint (2GB)
- **Accuracy Leader**: Viper achieves highest accuracy (89%) but demands 2.6x longer processing time (26.33s)
- **Performance Penalty**: Higher accuracy models require significantly more computational resources and processing time

Resource Intensity Analysis

- **Memory Scaling**: GPU usage ranges from 2GB (vonJack) to 10GB (Viper) - 5x difference
- **Processing Efficiency**: Token throughput varies from 9.50 (Viper) to 24.44 tokens/sec (vonJack)
- **Deployment Impact**: Resource requirements directly influence scalability and operational costs in production environments

Performance Trade-offs and Resource Efficiency

Performance and Efficiency Metrics



Optimal Balance Performers

- **Balanced Models:** EpsteinLama and AceCoder achieve 89% accuracy with moderate resource consumption
- **Practical Efficiency:** Both models offer 12-14 second response times with reasonable memory usage (3-6GB)
- **Production Viability:** These models provide enterprise-ready performance without extreme resource demands

Model Architecture Implications

- **Size-Performance Correlation:** Smaller models (vonJack) excel in speed while larger models (Viper) prioritize analytical depth
- **Consistent Output Quality:** Similar token generation (~4,500 tokens) across models suggests comparable explanation thoroughness
- **Strategic Selection:** Model choice depends on specific use case priorities - speed vs. accuracy vs. resource constraints

Model Speed Comparison

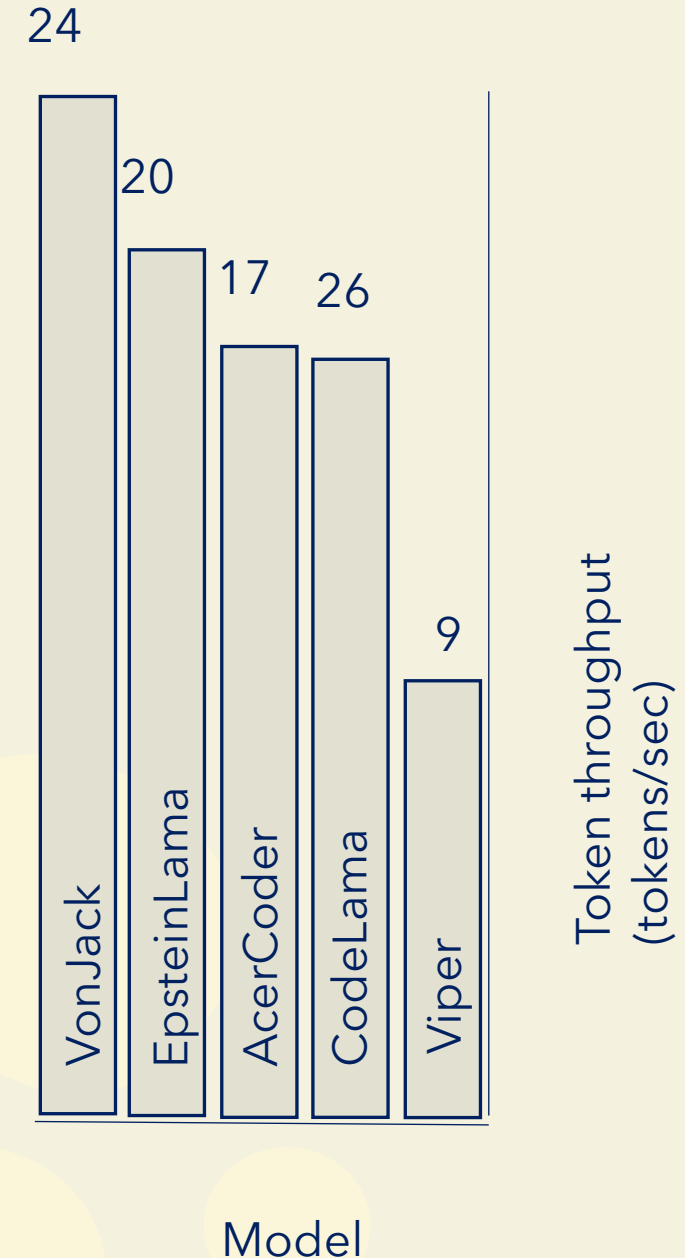
Comparing models token throughput

Fastest: vonJack (0.5B parameters) - 2.6x faster than Viper

Slowest: Viper (14B parameters) - highest accuracy cost

Balanced: AceCoder - good speed-accuracy trade-off

Correlation: Larger models = slower inference but potentially better results



LLM Prompt Response Quality

Input prompt:

```
class UserData(Serializable): def __init__(self, name, role): self.name = name self.role = role def readObject(self, stream): # Dangerous active deserialization pattern self.name = stream.readObject() self.role = stream.readObject() if self.role == "admin": self.grantAdminAccess()
```

Output Response:

VonJack (0.5B)

Functional issue 1: The `grantAdminAccess()` method is not defined anywhere. This will result in an `AttributeError` when trying to call it.

Corrected Code for Functional Issue:

```
```python
def grant_admin_access(self):
 print("Granting admin access")
```
```

Security Vulnerability Explanation:
The provided code snippet uses a dangerous active deserialization pattern. In Python's pickle module (which is used by default with `Serializable`), there can be potential attacks like arbitrary code execution through maliciously crafted serialized objects.

Viper (14B)

The provided Python class `UserData` has both potential security vulnerabilities and functional issues. Here's a breakdown of each:

Security Issues

1. **Active Deserialization**:
 - Using `stream.readObject()` to deserialize data can be dangerous because it allows arbitrary objects to be created from untrusted input.
 - This is particularly problematic when combined with granting administrative access based on serialized data.
2. **Granting Administrative Access**:
 - If an attacker manages to control or manipulate the serialized object (e.g., by setting `role` as `"admin"`), they could gain unauthorized administrative privileges.

Functional Issues

1. **No Validation for Role**:
 - There's no validation to ensure that only specific roles should have admin access; any string value set as `role` will grant admin rights.

Corrected Code

To address these concerns, we need to implement proper validation and use safer methods for serialization/deserialization.

Epstein (4B)

The provided Python class `UserData` has a potential security vulnerability related to serialization. The use of `readObject()` can lead to arbitrary object creation during deserialization, which is dangerous because it allows attackers to inject malicious objects.

Security Vulnerabilities Identified:

1. **Active Deserialization**: Using methods like `stream.readObject()` for reading serialized data directly into instance variables without proper validation or sanitization can allow an attacker to execute arbitrary code by sending specially crafted input.
2. **Lack of Input Validation**: There's no check on what gets assigned to `name` or `role`. An attacker could potentially set `role` as `'admin'` even when they shouldn't be allowed that access level.

Functional Issues Identified:

- If `grantAdminAccess()` method isn't defined elsewhere (not shown here), calling it would result in an `AttributeError`.

Impact of Prompting Techniques

Chain-of-Thought vs Zero-Shot Performance

Overall Findings

- ▶ Chain-of-thought prompting **significantly improved** performance
- ▶ Most dramatic improvements in smaller models
- ▶ Security detection more affected than functional validation

Model-Dependent Benefits

- ▶ **vonJack**: +56pp security, +27pp overall
- ▶ **EpsteinLama**: +34pp security, +22pp overall
- ▶ **CodeLlama**: +11pp security, +5pp overall
- ▶ **AceCoder/Viper**: No measurable impact

Performance Convergence

- ▶ Most models reached **78% security accuracy** with CoT
- ▶ Suggests similar reasoning capabilities when guided
- ▶ Eliminated false negatives in 3 models

Prompting Technique Analysis

Key Insight: Chain-of-Thought Prompting Levels the Playing Field

Smaller models showed the most dramatic improvements, while larger models demonstrated ceiling effects

Security Detection Improvements

- ▶ vonJack: 22% → 78% (+56 percentage points)
- ▶ EpsteinLama: 44% → 78% (+34 percentage points)
- ▶ CodeLlama: 67% → 78% (+11 percentage points)
- ▶ AceCoder/Viper: Already at ceiling (78%)

Functional Validation Impact

- ▶ Less dramatic improvements overall
- ▶ EpsteinLama: 89% → 100%
- ▶ Other models: Minimal or no change
- ▶ Already strong baseline performance

Cost-Benefit Analysis

- ▶ +252 input tokens per prompt (+17.4%)
- ▶ Minimal response time impact (<2%)
- ▶ Significant accuracy gains justify overhead
- ▶ Particularly valuable for smaller models

Why Chain-of-Thought Works

Structured Reasoning

- ▶ Forces systematic analysis approach
- ▶ Reduces random pattern matching
- ▶ Encourages consideration of edge cases
- ▶ Improves multi-step vulnerability detection

Model Architecture Implications

- ▶ Smaller models benefit most from guidance
- ▶ Larger models may have built-in reasoning
- ▶ Architecture differences affect responsiveness
- ▶ Training data quality influences ceiling effects

False Negative Reduction

- ▶ Eliminated FNs in CodeLlama, EpsteinLama, vonJack
- ▶ Improved recall from 57-86% to 100%
- ▶ Critical for security applications
- ▶ Consistent false positive patterns remain

Practical Implication: Chain-of-thought prompting can make smaller, resource-efficient models competitive with larger models for security analysis tasks

RELATIVE COMPARISON OF OPEN-SOURCE LLMs

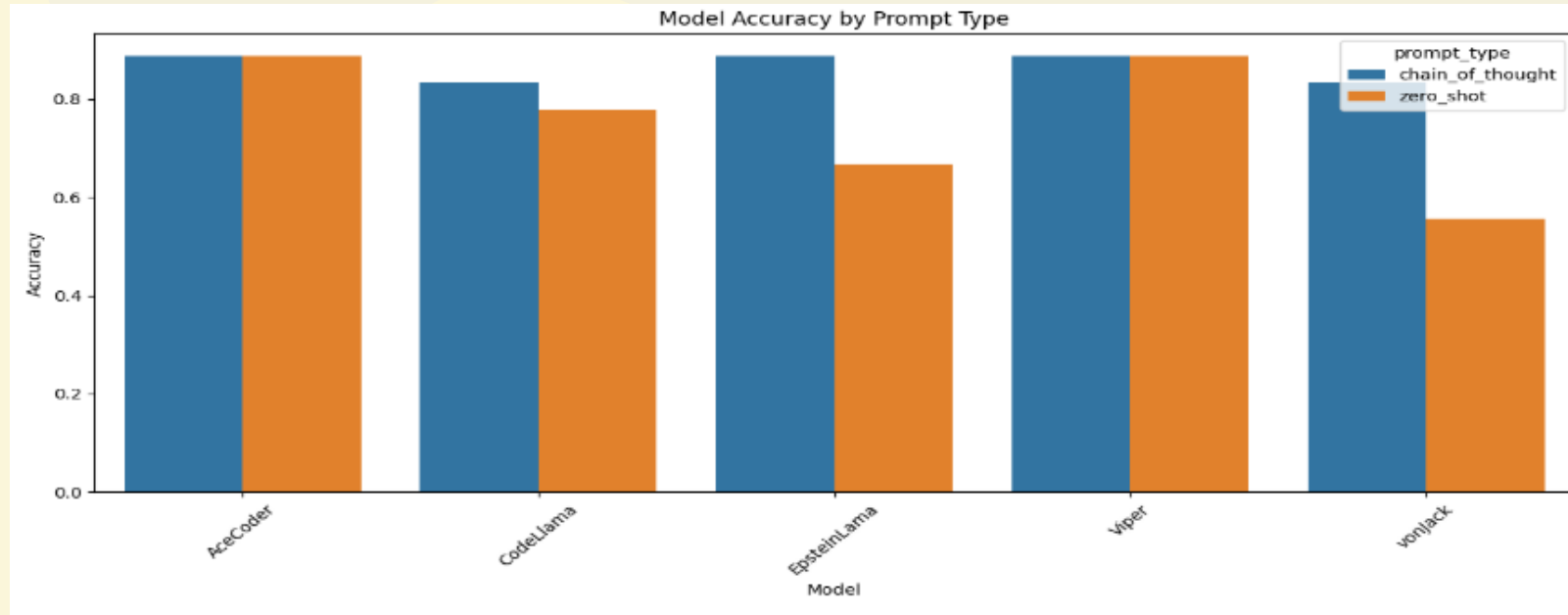
LLM Results findings

| Model | Parameters | Architecture | Training Focus | Key Strengths |
|-------------|------------|---------------|-----------------------|----------------------|
| vonJack | 0.5B | Qwen2.5-Coder | Code Generation | Speed, Efficiency |
| EpsteinLama | 4B | LLaMA 3.2 | Agent/Coder | Adaptability |
| CodeLlama | 8B | LLaMA 3.1 | Code Tasks | Balanced Performance |
| AceCoder | 7.6B | Qwen2.5 | Instruction Following | Consistency |
| Viper | 14B | Custom Coder | Complex Analysis | Deep Understanding |

- Smaller models offer better performance at the cost of accuracy
- Token throughput varies dramatically from 9.50 (Viper) to 24.44 tokens/sec (vonJack) allowing teams to parse large codebases to be evaluated for weaknesses and vulnerabilities
- Resource requirements directly influence scalability and operational costs as teams are limited by their access to sufficient hardware to run larger models
- Models fine tuned on different data sets perform better and perform better in certain tasks

RELATIVE COMPARISON OF OPEN-SOURCE LLMs

Overall precision of prompting technique



Prompting techniques can improve model accuracy where structured prompting eliminates gaps between models

Model accuracy scales with parameter size with larger model being able to operate unguided

Smaller models benefit from guided prompting being a more resource efficient option

Open source LLMs are able to identify security vulnerabilities and perform functional validation given the model is sufficiently large

RELATIVE COMPARISON OF OPEN-SOURCE LLMs

Functional and Security Vulnerability Detection Performance

| Model | Security Accuracy | Security True Positives | Security False Negatives | Functional Accuracy | Functional True Positives | Functional False Negatives | Overall Accuracy |
|-------------|-------------------|-------------------------|--------------------------|---------------------|---------------------------|----------------------------|------------------|
| AceCoder | 78% | 7/9 | 0/9 | 100% | 9/9 | 0/9 | 89% |
| Viper | 78% | 7/9 | 0/9 | 100% | 9/9 | 0/9 | 89% |
| CodeLlama | 67% | 6/9 | 1/9 | 89% | 8/9 | 1/9 | 78% |
| EpsteinLama | 44% | 4/9 | 3/9 | 89% | 8/9 | 1/9 | 67% |
| vonJack | 22% | 2/9 | 5/9 | 89% | 8/9 | 1/9 | 56% |

Model Reliability Patterns

Consistency Leaders: AceCoder and Viper maintain stable performance across both domains, making them most reliable for comprehensive code review

Specialized Strengths: vonJack excels at functional validation (89%) but struggles with security (22%), suggesting potential for task-specific deployment strategies

Improvement Potential: EpsteinLama's moderate security performance (44%) combined with strong functional results indicates significant room for enhancement with better prompting

Practical Deployment Implications

Hybrid Approach Necessity: Performance variations suggest combining multiple models or using specialized models for different analysis types rather than relying on single-model solutions

Risk Tolerance Matching: Organizations with high security requirements should prioritize AceCoder/Viper, while development teams focused on functional correctness could leverage faster models like vonJack

Human Oversight Requirements: The security detection limitations across all models emphasize the continued need for expert human review in security-critical applications



Model-Specific Strengths and Weaknesses

VonJack (0.5B)

- ✓ Strengths: Fastest response, lowest resource usage
- ✗ Weaknesses: Poor zero-shot security detection

- 🎯 Best Use: High-throughput functional validation, CI/CD integration
- 📈 Improvement: Massive gains with structured prompting (+56% security)

EpsteinLama (4B)

- ✓ Strengths: Highest improvement potential, balanced resource usage
- ✗ Weaknesses: Inconsistent zero-shot performance

- 🎯 Best Use: Adaptive systems, educational applications
- 📈 Improvement: Excellent response to better prompting (+34% security)

AceCoder (7.6B)

- ✓ Strengths: Consistent performance, good resource efficiency
- ✗ Weaknesses: Limited improvement potential (already optimised)

- 🎯 Best Use: Enterprise deployment, reliable code review
- 📈 Improvement: Stable excellence across prompting methods

Viper (14B)

- ✓ Strengths: Highest capacity, excellent complex analysis
- ✗ Weaknesses: Resource intensive, slower inference

- 🎯 Best Use: Critical security audits, complex vulnerability detection
- 📈 Improvement: Consistent top-tier performance

Practical Implications

Open-source LLMs assist code reviews by flagging common vulnerabilities like SQL injection.

They automate routine tests, reducing reviewer workload and allowing focus on complex threats.

Cost-effective and customizable, lowering barriers for smaller teams.

Chain-of-thought prompting enhances detection accuracy.

Strong functional validation helps catch logical flaws early.

Suitable for integration into CI/CD pipelines for early issue detection.

Enable hybrid workflows combining automation with expert oversight.

Study Limitations

Dataset Scope

- ▶ Focus on well-known vulnerabilities in OWASP Benchmark
- ▶ May not reflect emerging threats or complex real-world systems
- ▶ Limited to single-file code snippets
- ▶ Missing multi-file dependencies and cross-module interactions

Technical Constraints

- ▶ Computational limits excluded larger models (70B+ parameters)
- ▶ Google Colab resource constraints
- ▶ Java-specific dataset limits language generalizability
- ▶ Static analysis only - no dynamic behavior assessment

Evaluation Gaps

- ▶ Limited assessment of explanation quality and clarity
- ▶ No evaluation of remediation guidance effectiveness
- ▶ Missing analysis of ethical and operational risks
- ▶ Reproducibility concerns across different hardware/software

Addressing Limitations

Future Research Directions

Expanded Scope

- ▶ Multi-language evaluation (Python, C++, JavaScript)
- ▶ Dynamic runtime analysis integration
- ▶ Cross-file dependency detection
- ▶ Emerging vulnerability patterns

Enhanced Evaluation

- ▶ Larger model assessment (70B+ parameters)
- ▶ Explanation quality metrics
- ▶ Long-term workflow integration studies
- ▶ Human-AI collaboration frameworks

Hybrid Approaches

- ▶ Combine LLMs with traditional analysis tools
- ▶ Develop specialized security-focused models
- ▶ Create adaptive prompting strategies
- ▶ Integrate with CI/CD pipelines

Next Steps: Comprehensive multi-language benchmarks, real-world deployment studies, and hybrid human-AI security review frameworks

Conclusion

Main Finding

Open-source LLMs show **strong potential** for supporting code reviews, with chain-of-thought prompting significantly enhancing performance, especially in smaller models

Strengths

- ▶ Strong functional validation (89-100%)
- ▶ Effective vulnerability detection with proper prompting
- ▶ Cost-effective alternative to proprietary tools
- ▶ Transparency and customization benefits

Current Limitations

- ▶ Consistent false positives
- ▶ May miss subtle vulnerabilities
- ▶ Limited cross-module dependency understanding
- ▶ Not yet ready to replace expert analysis

Practical Impact

- ▶ Can support initial code reviews
- ▶ Reduce burden on human reviewers
- ▶ Enable broader adoption of AI-assisted security practices
- ▶ Foundation for hybrid human-AI workflows

Research Value

- ▶ First systematic evaluation of open-source LLMs for security
- ▶ Demonstrates power of prompting techniques
- ▶ Establishes baseline for future comparisons
- ▶ Identifies key areas for improvement



Thank You!
Any Questions